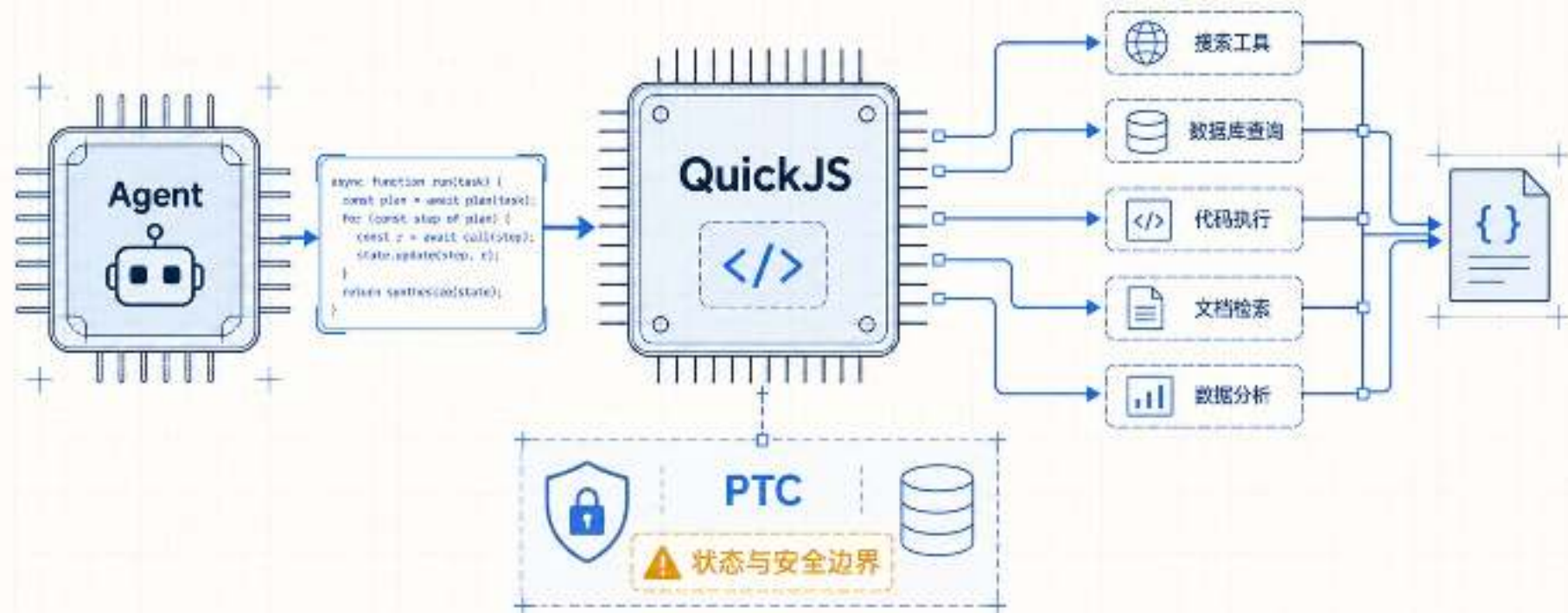


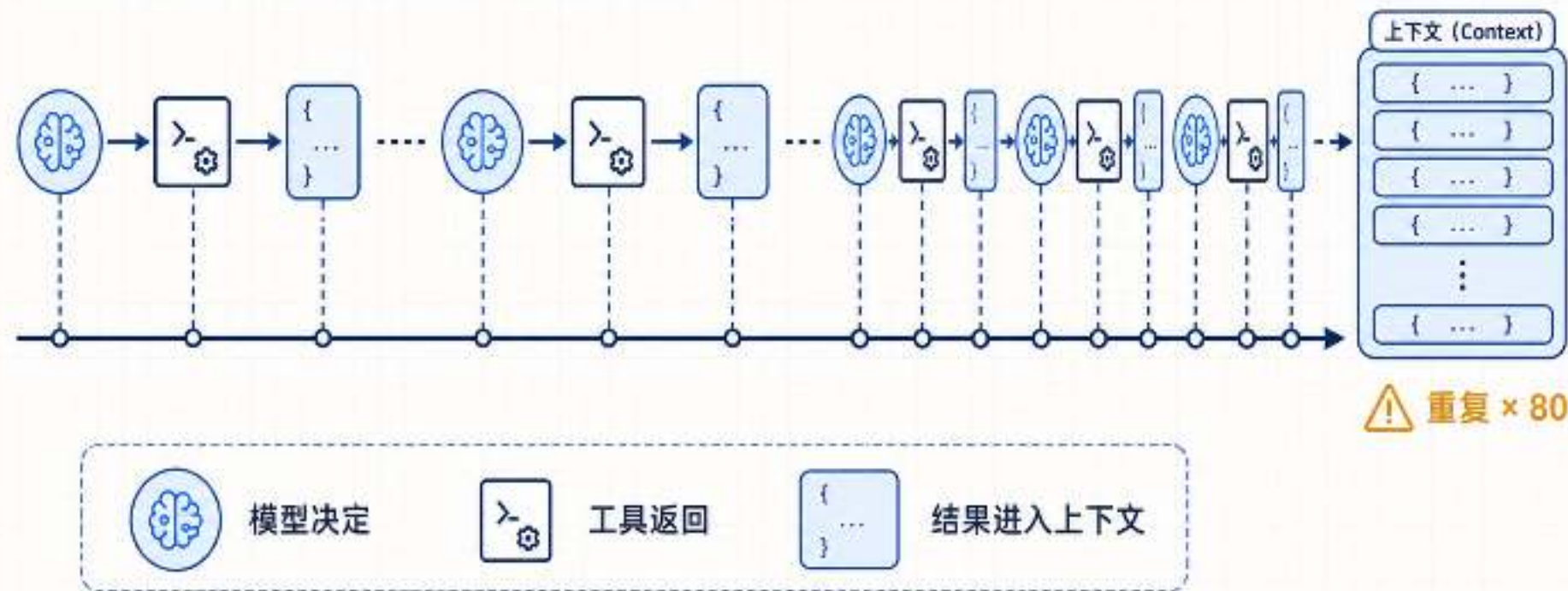
Interpreters: 让 Agent 用代码编排工具与数据

Deep Agents 实战 · 第 19 讲



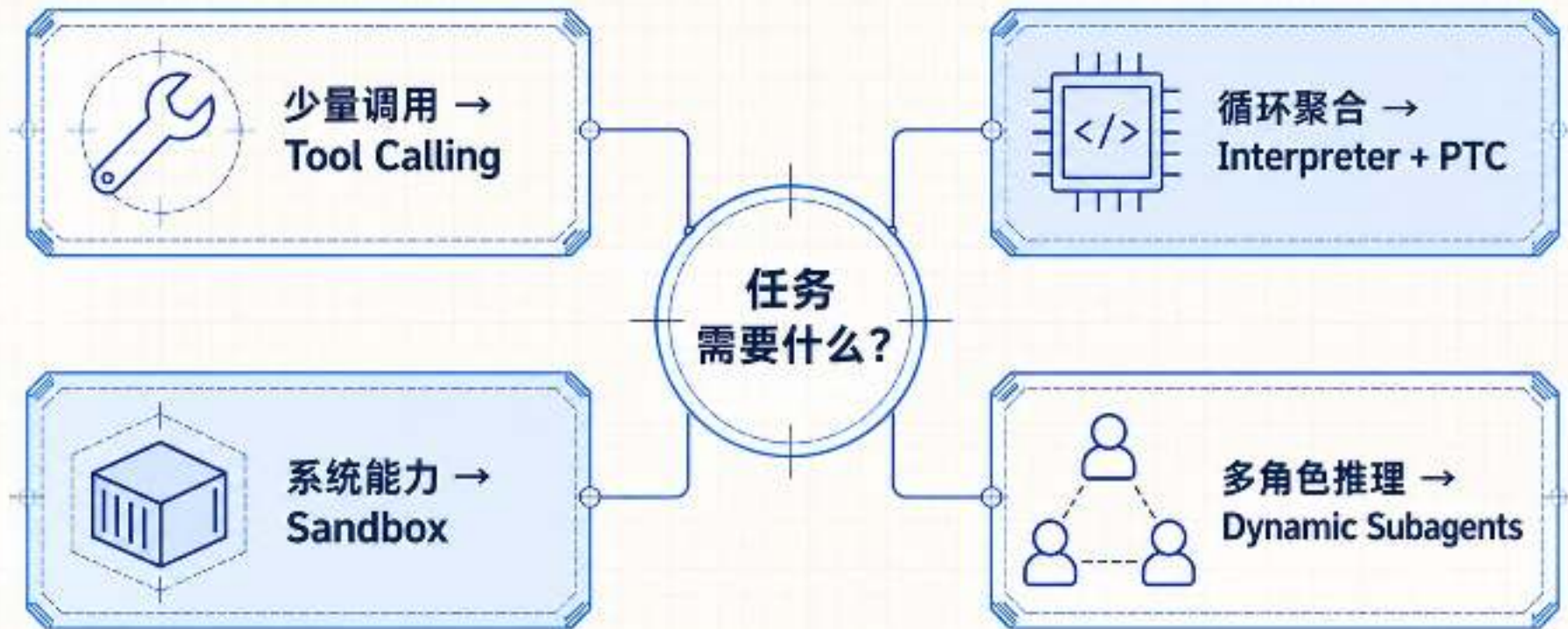
顺序工具调用的问题，不在工具本身

中间结果越多，模型上下文越拥挤



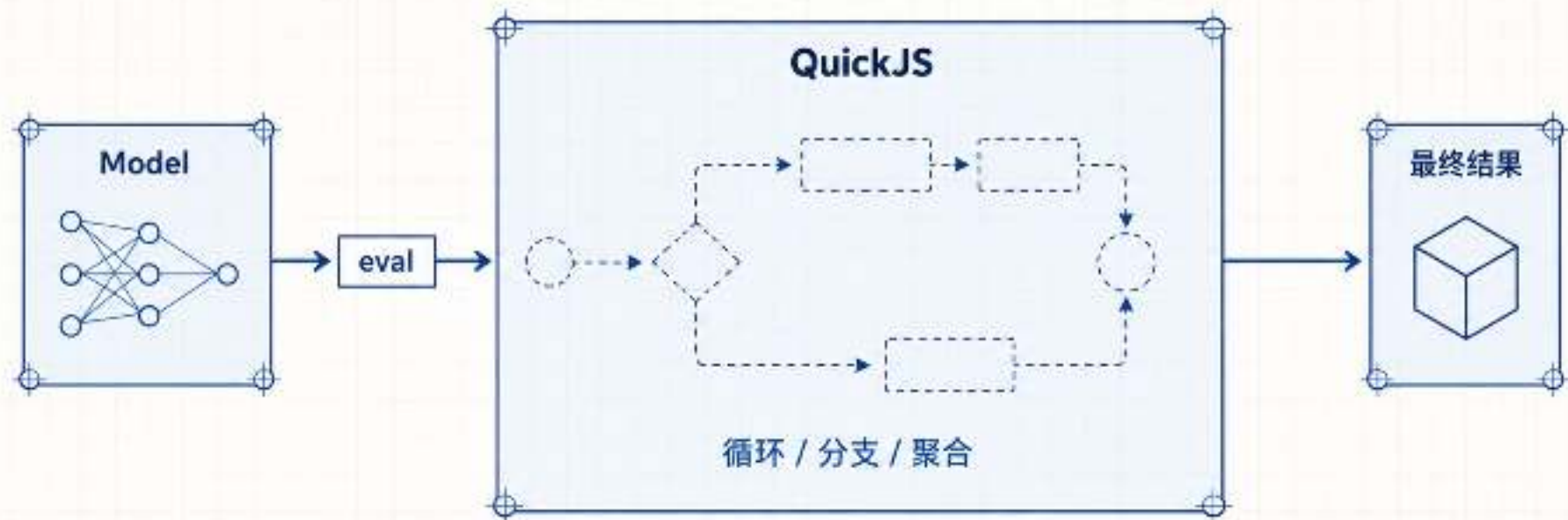
先看任务形状，再决定执行机制

调用规模、系统能力和推理角色是三个分岔点



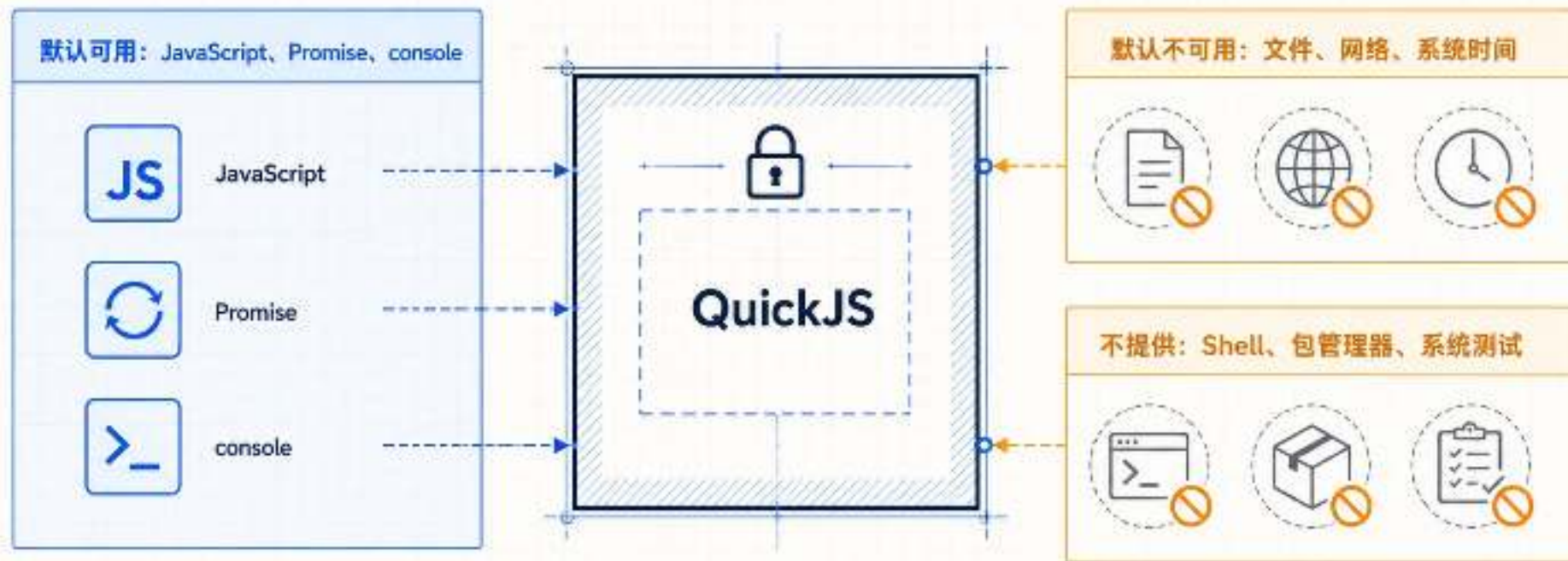
Interpreter 把确定性控制流移进 QuickJS

模型写 JavaScript，运行时执行循环、分支和聚合



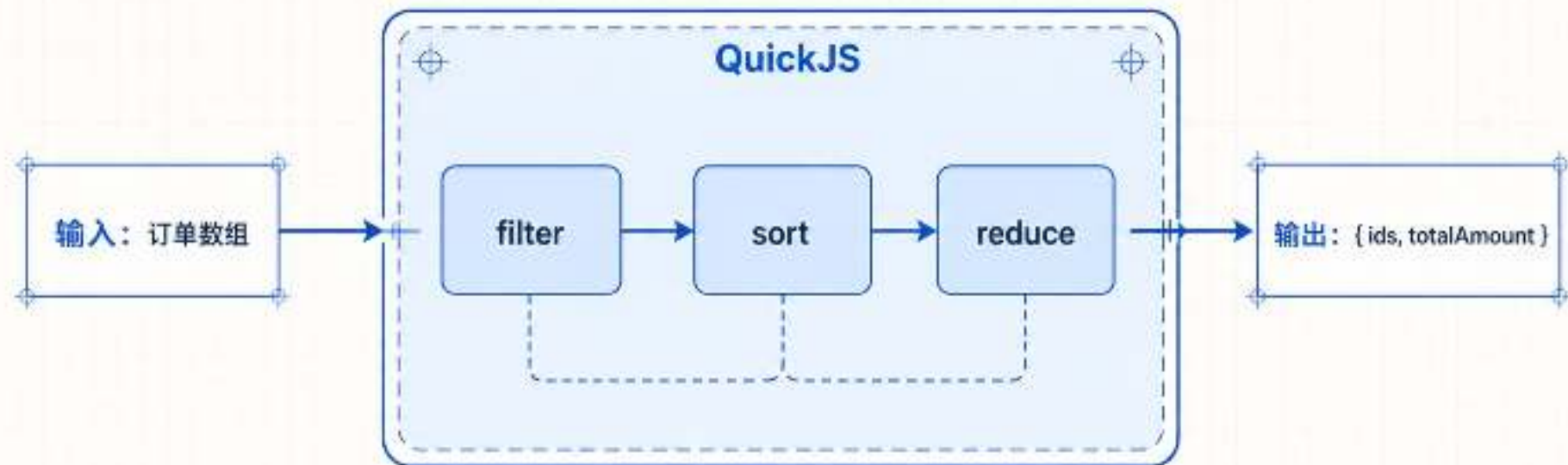
默认封闭，权限通过工具逐项打开

QuickJS 不是轻量版 Shell



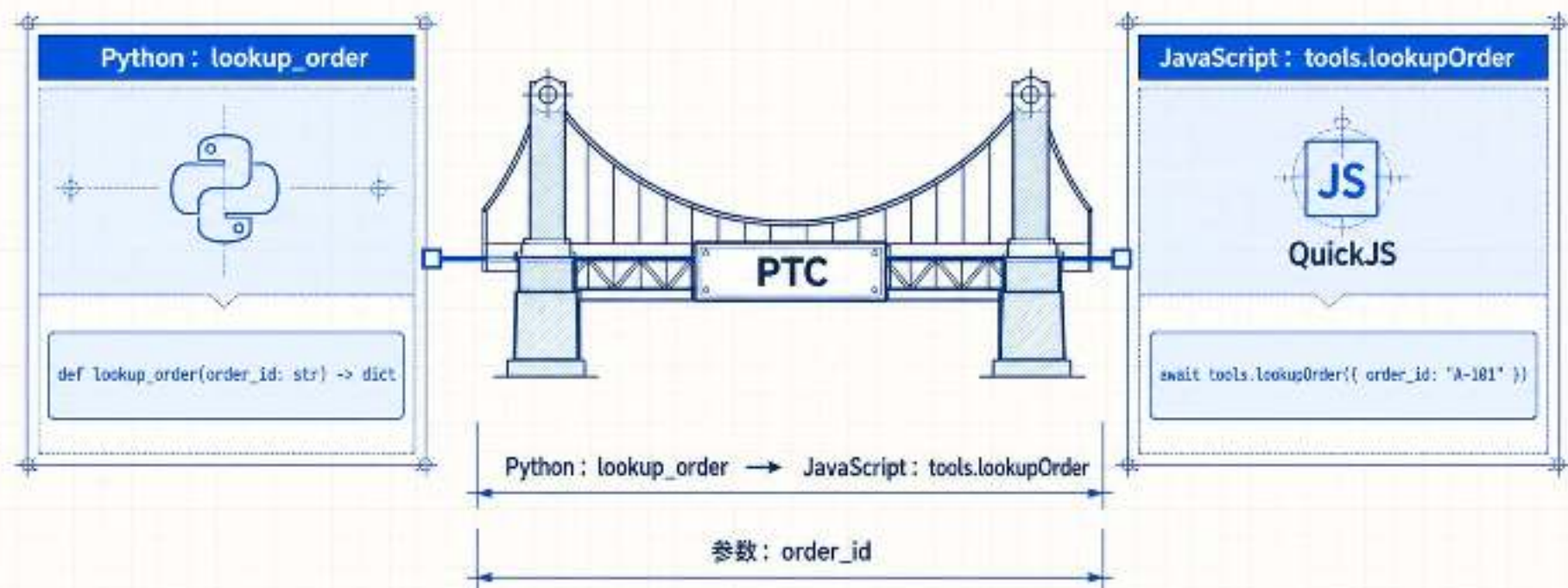
eval 只把最后一个表达式带回模型

排序、筛选和计数留在代码里完成



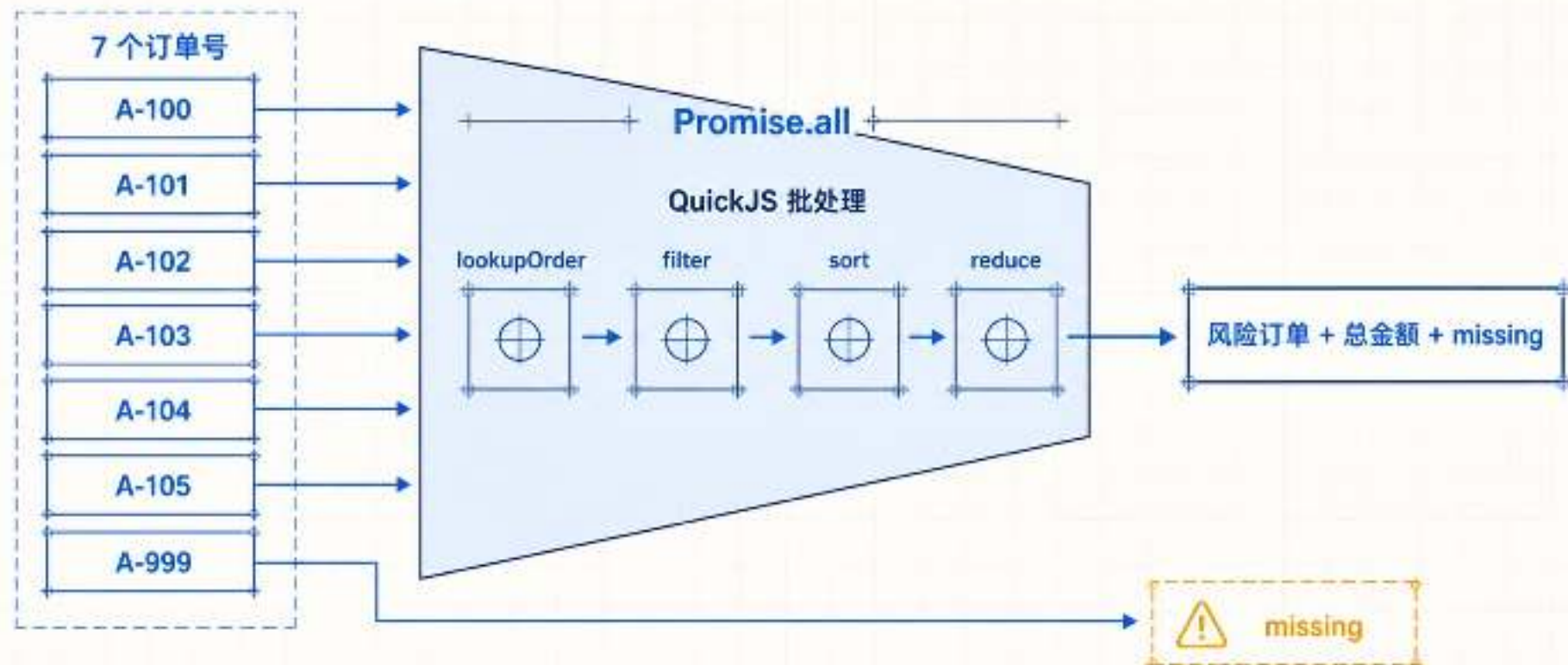
tools.lookupOrder 是 PTC 的桥

函数名转 camelCase，参数仍遵循 Python Tool Schema



并行查询留在代码，模型只收汇总结果

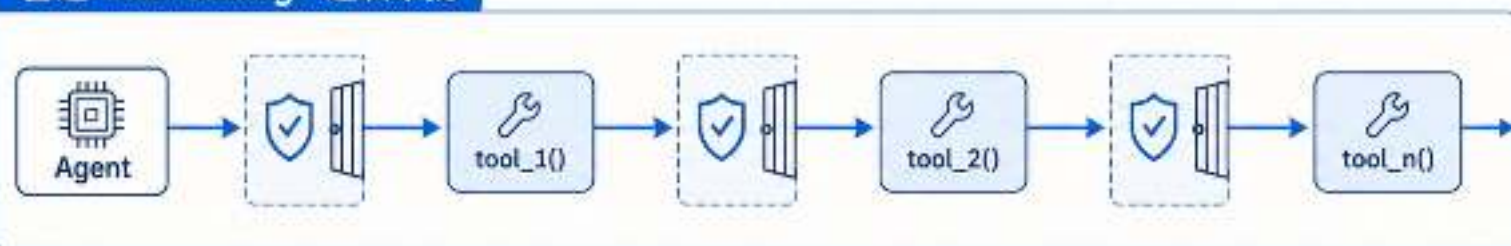
Promise.all 覆盖全部输入，缺失项单独归类



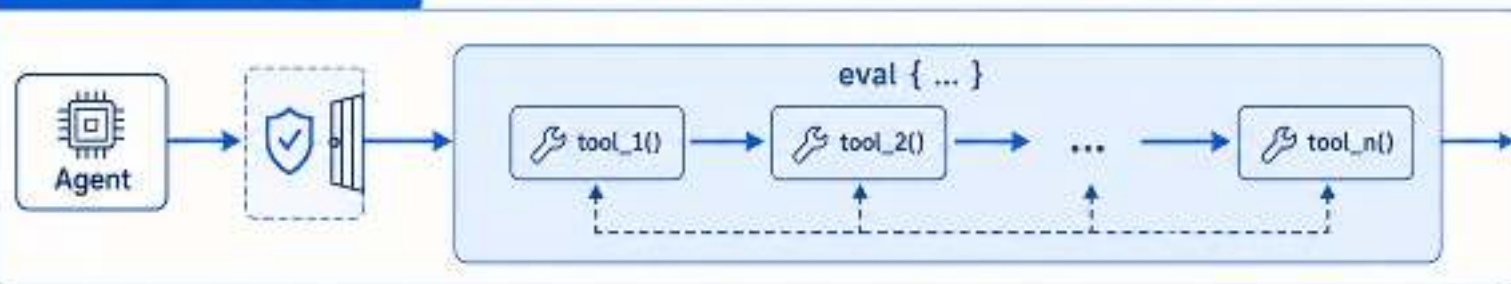
PTC 改变调用路径，也改变审批边界

interrupt_on 不会逐次覆盖 PTC 内部调用

普通 Tool Calling: 逐次审批



PTC: eval 内部批量执行



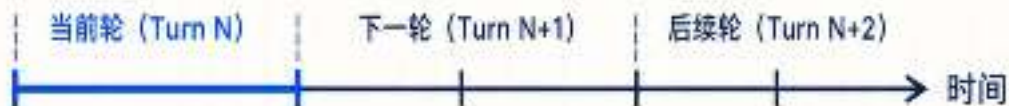
call、turn、thread 决定状态寿命

先选最短生命周期，再按任务需要放宽

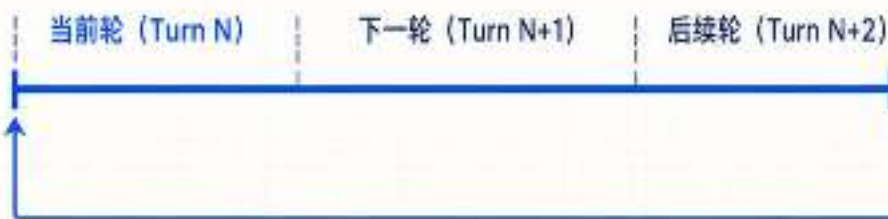
call: 一次 eval



turn: 当前 Agent turn



thread: 跨轮恢复



Snapshot

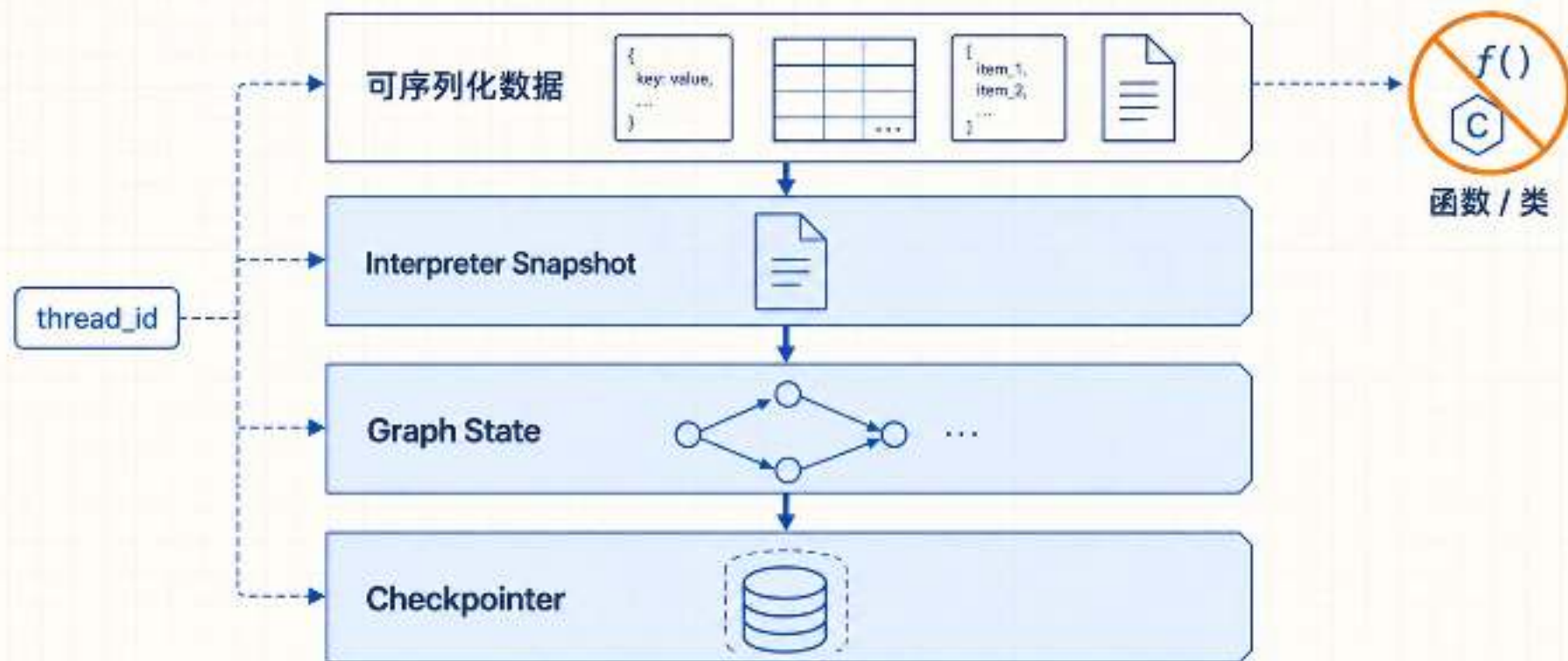


Checkpoint



Snapshot 保存数据, Checkpointer 保存 Graph State

跨轮复用同一个 thread_id





预算要按合法批次收紧

运行时默认值不是业务建议值

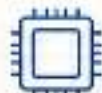


timeout

5



timeout: 默认 5 秒



memory_limit

64



memory_limit: 默认 64 MB



max_ptc_calls

256

16



max_ptc_calls: 默认 256, 订单实验 16



max_result_chars

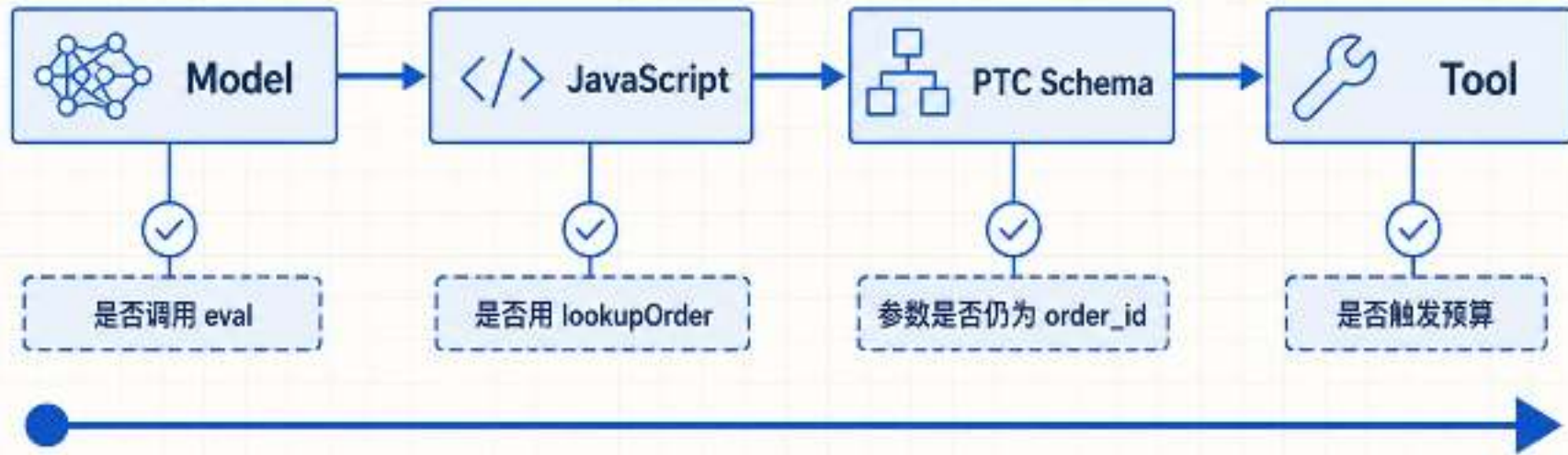
4000



max_result_chars: 默认 4000

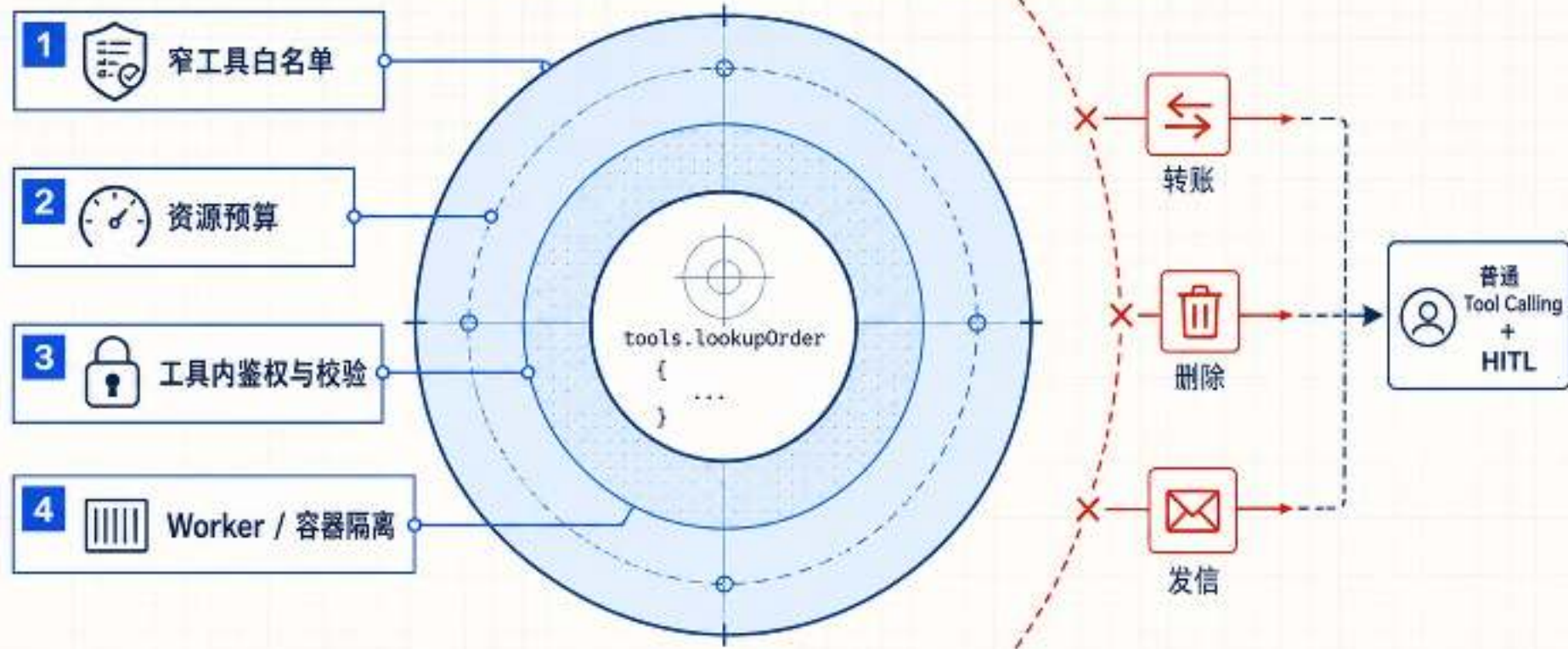
沿调用链排错，别只改 Prompt

Model → JavaScript → PTC Schema → Tool



PTC 安全靠四层边界，不靠一个开关

工具能做什么，解释器代码就能做什么



把 Interpreter 用在确定性编排上

模型决定目标，代码负责循环、分支和聚合



任务形状匹配



PTC 白名单最小



状态范围够短



预算与审批清晰

四项全通过，才能上线执行。